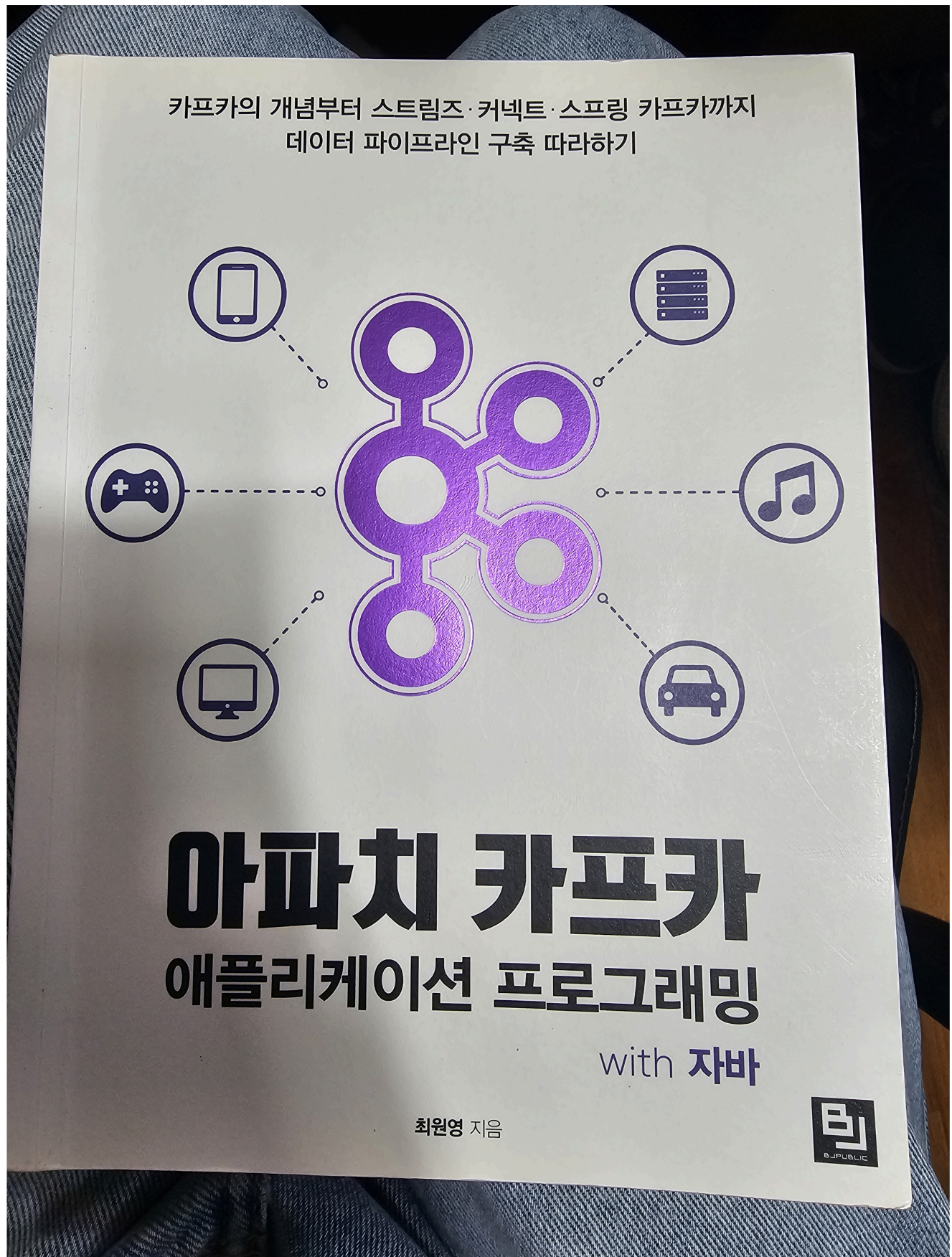


ch01 카프카란

0. 읽기 전



이 책을 기반으로 카프카 개념에 대하여 간단히 공부하였다.

Spring boot 공부를 하는데 JMS, RabbitMQ, 카프카가 나오게 되며 카프카에 대해서 먼저 간단하게 공부 해보는 시간을 갖게 되었다.

<https://plusb3b.kr/pdf-download-page/download.html>

위 주소는 내가 직접 필기한 것을 바탕으로 작성한 카프카 내용이다.

보기에는 어렵지만 그래도 도움이 된다면 다운로드를 추천한다.

1. 카프카란 무엇인가?

Kafka는 **대용량 이벤트 스트리밍 플랫폼**이다.

JMS는 자바 표준 API 사양이고, RabbitMQ는 AMQP 기반 메시지 브로커인 반면, 카프카는 **분산 이벤트 스트리밍에 최적화된 플랫폼**이다.

카프카는 **토픽(Topic) → 파티션(Partition)** 순서대로 메시지를 기록하며, **소비자(Consumer)**는 **오프셋(Offset)**을 스스로 관리하면서 읽기를 진행한다.

우선 카프카의 특징을 간략하게 알아보고, 후에 자세히 다시 설명하겠다.

1.1 카프카의 주요 특징

고처리량 / 수평확장

- 파티션 단위 병렬 처리로 대용량 데이터를 효율적으로 처리한다
- 브로커 추가를 통한 수평 확장이 용이하다

내구성 / 재처리

- 디스크에 메시지를 **지속 저장하며 복제**한다
- **오프셋 조정**을 통해 과거 이벤트 재처리가 가능하다

순서 보장

- 파티션 내에서 메시지 순서를 보장한다
- 동일 파티션 내 메시지는 **작성된 순서대로** 읽힌다

보관 정책

- 시간 또는 용량 기반 보관 정책을 설정할 수 있다
- 로그 컴팩션(Log Compaction)을 통한 최신 상태 유지가 가능하다

전달 보장

- 기본값은 **At-least-once**(최소 한 번, 중복 가능)이다

- 트랜잭션을 사용하면 **Exactly-once**(정확히 한 번) 처리도 가능하다

주요 용도

- 실시간 로그/이벤트 수집
 - CDC(Change Data Capture)
 - 스트림 처리(Stream Processing)
 - 마이크로서비스 간 비동기 통신
-

1.2 Kafka vs JMS vs RabbitMQ 비교

1) 본질

- **Kafka**: 분산 이벤트 스트리밍 플랫폼
- **JMS**: Java 표준 메시징 API 사양
- **RabbitMQ**: AMQP 기반 메시지 브로커

2) 프로토콜

- **Kafka**: 카프카 전용 바이너리 프로토콜
- **JMS**: 구현체(브로커)에 따라 상이함
- **RabbitMQ**: AMQP, STOMP, MQTT 등 다양한 프로토콜 지원

3) 메시지 모델

- **Kafka**: Pub/Sub 모델(토픽-파티션 구조), 소비자가 **Pull 방식**으로 메시지를 가져온다
- **JMS**: Queue(P2P) / Topic(Pub/Sub) 모델
- **RabbitMQ**: Queue 중심, Exchange를 통한 라우팅, 브로커가 **Push 방식**으로 메시지를 전달한다

4) 순서 보장

- **Kafka**: 파티션 내에서 순서를 보장한다
- **JMS**: 구현체에 따라 다르다
- **RabbitMQ**: 단일 소비자의 경우 Queue 내 순서 보장, 다중 소비자 또는 재큐잉(Requeue) 시 순서가 깨질 수 있다

5) 처리량 / 확장성

- **Kafka**: 매우 높은 처리량, 수평 확장이 용이하다
 - **JMS**: 구현체에 따라 다르다
 - **RabbitMQ**: 높은 처리량을 제공하나, 복잡한 라우팅 패턴에 최적화되어 있다
-

6) 보관 / 재처리

Kafka

- **보관 기간**: 설정에 따라 **장시간 보관**이 가능하다 (기본 7일, 필요시 무제한 설정 가능)
- **재처리**: 컨슈머 오프셋을 조정하여 **언제든 과거 메시지 재처리**가 가능하다
- **특징**: 로그 기반 저장소로 동작하여 메시지가 소비되어도 **삭제되지 않는다**

JMS

- **보관 기간**: 일반적으로 브로커가 **짧은 시간만** 보관한다
- **재처리**: 구현체에 따라 다르지만, 일반적으로 재처리가 **제한적**이다
- **특징**: 메시지가 소비되면 **큐에서 제거**되는 것이 기본 동작이다

RabbitMQ

- **보관 기간**: 메시지가 소비되면 큐에서 **즉시 삭제**된다
 - **재처리**: **Dead Letter Queue(DLQ)**를 활용한 재처리가 가능하다
 - **특징**: **메시지 확인 응답(ACK)** 메커니즘을 통해 재전송을 지원한다
-

7) 라우팅

Kafka

- **방식**: 메시지 키를 **해시**하여 특정 파티션으로 라우팅한다
- **특징**: 단순하지만 효율적이며, **같은 키를 가진 메시지는 항상 같은 파티션**으로 전송된다
- **순서 보장**: 파티션 내에서 메시지 순서가 보장된다

JMS

- **방식**: 구현체(ActiveMQ, IBM MQ 등)에 의존한다
- **특징**: **Message Selector**를 통한 필터링이 가능하다
- **유연성**: 구현체마다 제공하는 라우팅 기능이 상이하다

RabbitMQ

- **방식**: **Exchange + Routing Key/Binding**을 통한 유연한 라우팅이 가능하다
 - **Exchange 타입**: Direct, Topic, Fanout, Headers 등 다양한 패턴을 지원한다
 - **특징**: 복잡한 메시지 라우팅 시나리오에 최적화되어 있다
-

8) 지연시간 (Latency)

Kafka

- **지연시간**: **매우 짧다** (밀리초 단위)

- **최적화 요소:**
 - 배치 전송으로 네트워크 효율성 극대화
 - 압축을 통한 데이터 전송량 감소
 - **Zero-copy** 기술로 빠른 데이터 전송
- **적합한 용도:** 대용량 실시간 데이터 스트리밍, 로그 집계

JMS

- **지연시간:** 구현체에 따라 다르다
- **특징:** 일반적으로 밀리초~초 단위의 지연시간을 보인다
- **적합한 용도:** 엔터프라이즈 메시징, 트랜잭션 처리

RabbitMQ

- **지연시간:** 낮은 편이다 (밀리초 단위)
- **특징:** 복잡한 라우팅에도 **효율적인 메시지 전달**이 가능하다
- **적합한 용도:** 업무 메시징, 작업 큐(Task Queue), 복잡한 라우팅 패턴이 필요한 경우

1.3 언제 무엇을 쓸까?

1) Kafka를 선택해야 하는 경우

- 마이크로서비스 이벤트 소싱/CQRS
- 로그/이벤트 수집 → 분석 파이프라인 구축
- 재처리가 중요할 때
- DB 변경사항 스트리밍, 실시간 대시보드/머신러닝 피드

2) RabbitMQ를 선택해야 하는 경우

- 작업 큐, 요청/응답 패턴
- 복잡한 라우팅(Topic/Headers/Direct)
- 즉시성 알림
- 메시지 패턴이 다양하고 세밀한 제어가 필요할 때

3) JMS를 선택해야 하는 경우

- 조직에 **JMS 기반 미들웨어**가 이미 표준일 때

2. 카프카 탄생

링크드인(LinkedIn)에서 파편화된 데이터 수집/분배 아키텍처를 운영하는 데 어려움을 겪으면서 시작되었다.

데이터 생성 및 적재 시 **데이터 생성을 담당하는 소스 애플리케이션**과 **데이터 적재를 담당하는 타깃 애플리케이션**이 직접 연결되어 있었다.

초기 운영 시 **단방향 통신**을 통해 소스 → 타깃으로 연동하였으며, 아키텍처가 복잡하지 않아 운영이 힘들지 않았다.

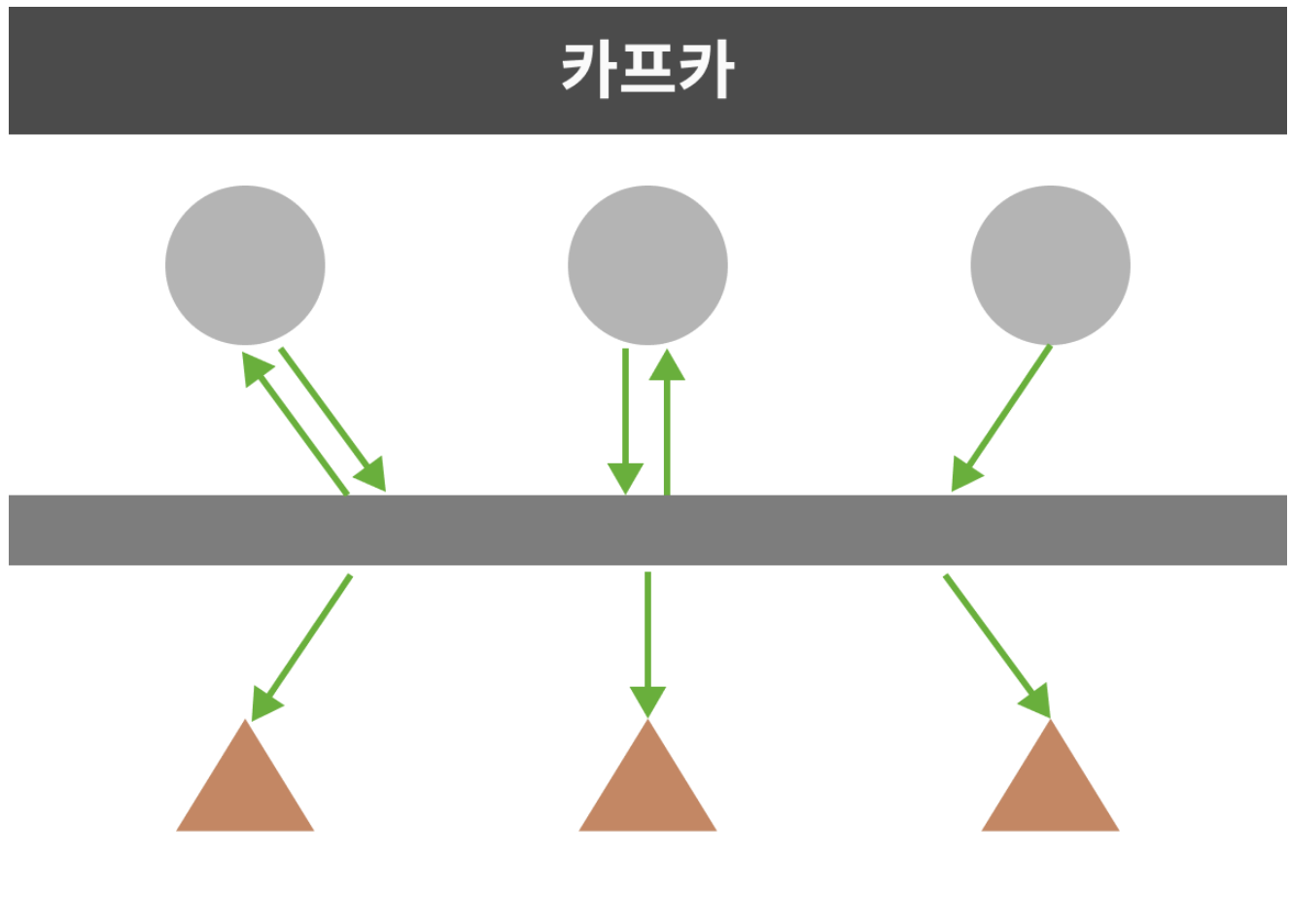
시간이 지나 아키텍처가 거대해지면서 소스/타깃 애플리케이션 개수가 증가하게 되었다. 이로 인해 **데이터 전송 라인이 기하급수적으로 많아졌다**.

소스 애플리케이션과 타깃 애플리케이션의 파이프라인 개수가 많아지면서 **버전 관리 이슈**가 생겼으며, 타깃에 장애가 발생하면 **소스에도 영향**이 가게 되었다.

위와 같은 이유들로 인해 서비스를 **안정적으로 운영하기 힘든 상황**에 직면하게 되었다.

기존 아키텍처를 개선하려는 시도들이 있었으나 큰 성과를 얻지 못했고, 결국 **신규 시스템을 개발**하게 되었다.

이것이 바로 **아파치 카프카(Apache Kafka)**이다. 아파치 카프카는 데이터를 **한 곳에 모아 처리할 수 있게 중앙 집중화**했다.



2.1 카프카를 중앙에 배치

소스와 타깃 사이의 의존도를 최소화

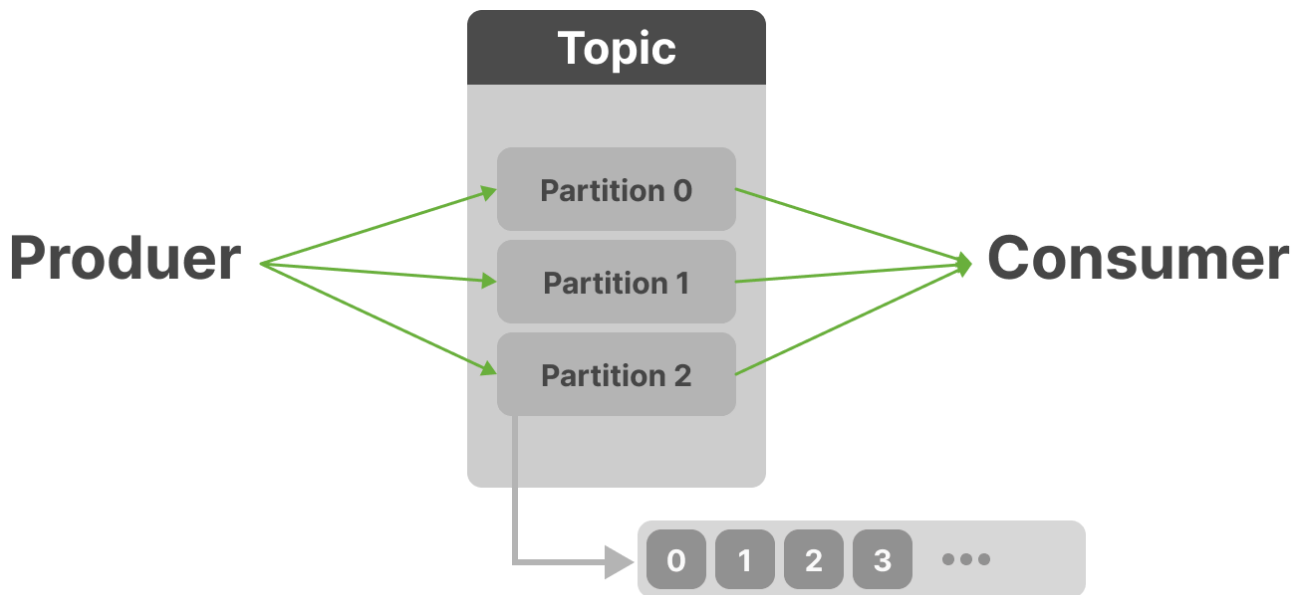
기존 **1:1 방식**은 한쪽의 이슈가 다른 쪽에 직접적인 영향을 주었다. 카프카를 중앙에 배치함으로써 이러한 **강한 결합을 느슨한 결합**으로 변경할 수 있었다.

데이터를 어디로 보낼지 고민하지 않아도 된다

일단 카프카에 넣으면 된다. 프로듀서는 데이터를 카프카에 전송하기만 하면 되고, 어떤 컨슈머가 사용할지 신경 쓰지 않아도 된다.

내부 구조

- 내부 데이터 저장 방식은 **FIFO(선입선출)** 구조이다
- Queue에 데이터를 보내는 것은 **프로듀서(Producer)** 역할이다
- Queue에서 데이터를 가져오는 것은 **컨슈머(Consumer)** 역할이다



데이터 포맷

데이터 포맷에는 사실상 제한이 없다. **직렬화/역직렬화**를 통해 **ByteArray**로 통신을 하게 된다.

카프카 클라이언트는 **ByteArray, ByteBuffer, Double, Long, String** 타입에 대응하는 **직렬화/역직렬화** 클래스를 제공한다.

카프카의 분산 운영

상용 환경에서의 카프카는 **최소 3대 이상의 서버(브로커)**에서 분산 운영을 한다.

- 프로듀서를 통해 전송받은 데이터를 파일 시스템에 안전하게 기록한다
- 카프카 클러스터 중 일부 서버에 장애가 있어도 데이터를 지속적으로 복제하여 안전하게 운영한다
- 데이터를 배치 전송을 통해 낮은 지연과 높은 데이터 처리량을 갖는다

배치 전송: 데이터를 묶음 단위로 처리하는 방식

데이터 레이크

빅데이터 저장/활용을 위해 생성되는 데이터를 모두 모아야 하는데, 이때 **데이터 레이크(Data Lake)** 개념이 사용된다.

데이터 레이크: 필터링과 패키지화가 되지 않은 원천 데이터를 저장하고 운영되는 서비스로부터 수집 가능한 데이터를 모으는 개념이다.

데이터를 데이터 레이크로 모으는 방법

엔드 투 엔드(End-to-End) 방식

- 데이터를 직접 데이터 레이크에 전송하는 방식
- 서비스가 비대해지고 복잡해지면 **카프카 도입 전과 같은 문제**가 발생한다

데이터 파이프라인 구축

- 데이터를 **추출(Extract) → 변경(Transform) → 적재(Load)** 과정을 묶어 데이터 파이프라인을 구축한다
- **안정성**을 추구하며 **유연하면서 확장 가능하게 자동화**한다

그럼 여기서 앞서 말한 **카프카의 특징**에 대해서 다시 되돌아볼 수 있다.

2.2 카프카의 특징과 왜 데이터 파이프라인으로 적합할까?

1) 높은 처리량

프로듀서가 브로커에 데이터를 보낼 때와 컨슈머가 브로커로부터 데이터를 받을 때 모두 묶어서 전송한다.

동일한 양의 데이터를 보낼 때 **네트워크 통신 횟수를 줄인다**. 동일 시간 내 **더 많은 데이터를 전송**할 수 있는 것이다.

파티션 단위를 통해 동일 목적의 데이터를 여러 파티션에 분배하며 데이터를 **병렬 처리**한다. 파티션 개수만큼 컨슈머를 늘려 **동일 시간당 데이터 처리량을 늘린다**.

2) 확장성

가변적인 환경에서 **안정적으로 확장** 가능하도록 설계되었다.

if) 데이터가 적을 때

카프카 클러스터 **브로커를 최소한의 개수로 운영한다.**

Scale-out (스케일 아웃)

if) 데이터가 많아질 때

클러스터 **브로커 개수 늘린다**

Scale-in (스케일 인)

다시 적어지면 개수 줄인다

모두 **무중단 운영**을 지원하여 안정적이다.

3) 영속성

영속성(Persistence): 데이터를 생성한 프로그램이 종료되더라도 사라지지 않는 데이터의 특성

카프카는 전송받은 데이터를 **파일 시스템**에 저장한다. OS 레벨에서 파일 시스템을 활용하는 방법을 찾게 되었다.

OS에서는 **파일 I/O 성능 향상**을 위해 **페이지 캐시 영역**을 메모리에 따로 생성하여 사용한다.

페이지 캐시 메모리 영역을 사용하여 **한 번 읽은 파일 내용은 메모리에 저장했다가 다시 사용**한다. 이것이 파일 시스템을 사용해도 **처리량이 좋은 이유**이다.

디스크 기반 파일 시스템을 활용하여 **애플리케이션 장애가 발생해도 프로세스를 재시작하여 안전하게 데이터를 처리**한다.

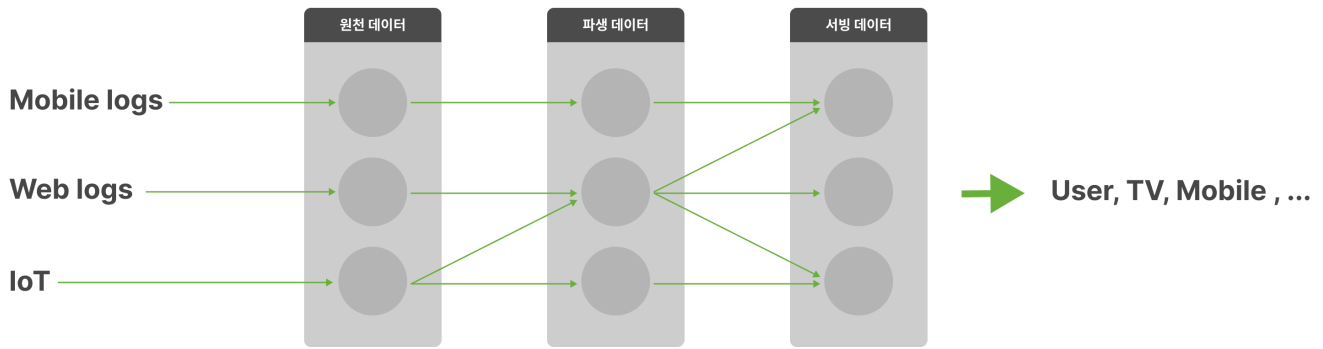
4) 고가용성

3개 이상의 서버들로 운영되는 카프카 클러스터는 **일부 서버에 장애가 발생해도 무중단으로 안전하고 지속적으로 데이터를 처리**한다.

프로듀서로부터 전송받은 데이터를 **다른 브로커에 복제**하여 저장한다. **한 브로커에 장애가 발생해도 복제되어 있어 지속적으로 데이터 처리가 가능하다.**

2.3 데이터 레이크 아키텍처와 카프카의 미래

레거시 데이터 플랫폼 아키텍처



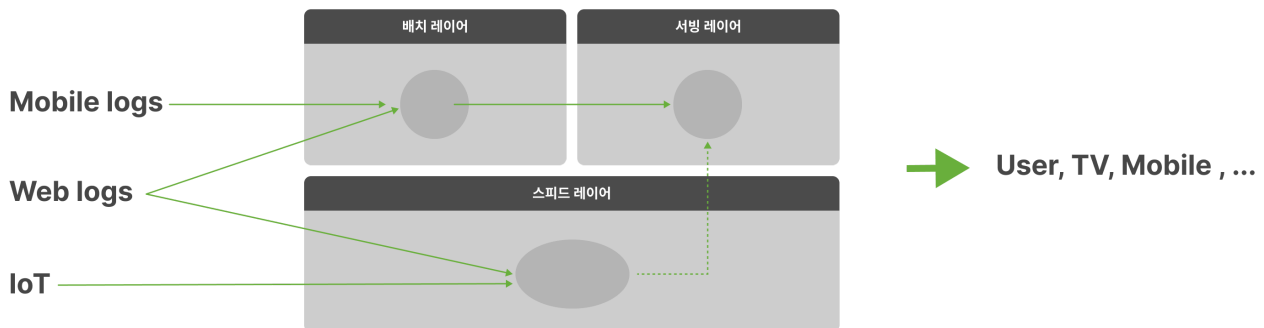
초기 빅데이터 플랫폼은 **엔드 투 엔드**로 각 서비스 애플리케이션으로부터 데이터를 **배치**로 모았다.

이는 다음과 같은 특징들을 갖고 있다:

- **유연하지 못함**
- 실시간으로 생성되는 데이터들에 대한 **인사이트**를 서비스 애플리케이션에 **빠르게 전달**하지 못함
- **파생된 데이터의 히스토리 파악이 어려움**
- 계속되는 데이터의 가공으로 인해 **데이터가 파편화**되어 데이터 표준 정책을 지키기 어려움

이를 해결하기 위해 기존 배치 데이터를 처리하는 부분 외에 **스피드 레이어(Speed Layer)**라 불리는 **실시간 데이터 ETL 작업 영역**을 정의한 아키텍처를 만들었다.

람다 아키텍처



주요 레이어

배치 레이어 (Batch Layer)

배치 데이터를 모아 **특정 시간, 타이밍마다 일괄** 처리한다.

서빙 레이어 (Serving Layer)

가공된 데이터를 데이터 사용자, 서비스 애플리케이션이 **사용할 수 있도록** 데이터가 저장된 공간이다.

스피드 레이어 (Speed Layer)

- 서비스에서 생성되는 **원천 데이터**를 **실시간으로 분석**하는 용도
- 배치 데이터에 비해 **낮은 지연**으로 분석이 필요한 경우 스피드 레이어를 통해 데이터를 분석한다

2.3.1 람다 아키텍처 (Lambda Architecture)

람다 아키텍처에서 카프카는 스피드 레이어에 위치한다.

데이터를 배치 처리하는 레이어와 실시간 처리하는 레이어를 분리한 람다 아키텍처는 데이터 처리 방식을 명확히 나눌 수 있다.

하지만 레이어가 2개로 나뉘기 때문에 다음과 같은 단점이 발생한다:

- 데이터 분석, 처리하는 데 필요한 로직이 각 레이어에 따로 존재한다
- 실시간 데이터와 배치 데이터를 융합하여 처리할 때 다소 유연하지 못한 파이프라인이 생성된다

이러한 람다 아키텍처의 단점을 해소하기 위해 카파 아키텍처(Kappa Architecture)를 제안하였다.

이는 배치 레이어를 제거하고 모든 데이터를 스피드 레이어에 넣어 처리한다.

2.3.2 왜 2개의 레이어가 단점일까?

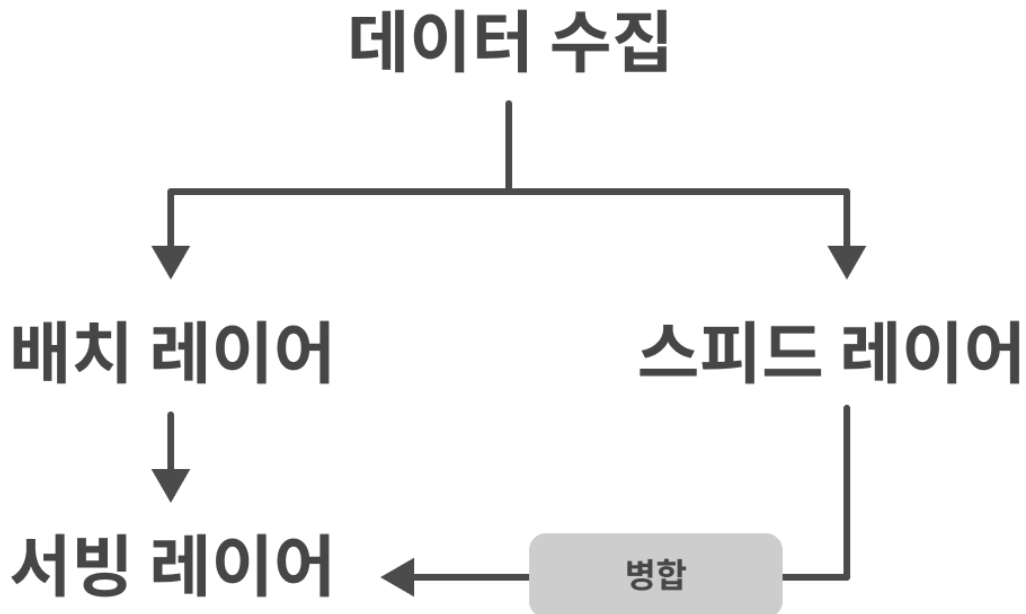
일반적으로 "관심사 분리"는 좋은 원칙이다.

- 배치 처리: 대량 데이터, 정확성 기준
- 스트림 처리: 지연 최소화, 즉시 응답 위주

목표가 다르니 분리하는 것은 설계 원칙상 자연스럽다.

그런데 람다 아키텍처의 분리는 단순한 역할의 분리가 아니다. 람다 아키텍처에서 말하는 분리는 코드와 데이터 흐름 자체를 2개로 복제하는 구조이다.

람다 아키텍처



각 레이어는 서로 다른 코드, 서로 다른 데이터 저장소, 서로 다른 처리 방식을 가진다. 한 기능을 두 번 구현해야 하는 상황이다.

람다 아키텍처의 실시간 + 배치 분석

람다 아키텍처는 실시간 + 배치 데이터 분석을 한다.

- **스피드 레이어**: 지금 막 들어온 데이터를 **빠르게 계산**
- **배치 레이어**: 모든 누적 데이터를 **정확히 계산**

계산 로직은 대부분 **동일한 경우가 많다**. 다음과 같은 예시를 보며 살펴보자.

예시: 실시간 매출 총합 계산

사용자가 결제할 때마다 **실시간 매출 합계**를 보고 싶고, 매일 밤 하루 총 매출을 **정확히 다시 계산**하고 싶은 상황 (매출 총합 구하기)

람다 구조에서의 구현

Speed Layer (실시간)

- Kafka로 들어온 이벤트를 **Flink/Storm**이 소비해 `sum(amount)` 계산
- **Redis** 같은 캐시 DB에 즉시 반영

Batch Layer (배치)

- 하루 치 데이터를 **HDFS**에 저장
- **Spark/Hadoop**으로 다시 `sum(amount)` 계산

- Hive나 Cassandra에 최종 결과 저장

결국 매출 합계 계산 로직을 각자 구현해야 한다.

사용하는 언어/프레임워크/저장소가 다르며, 같은 기능을 2개의 다른 환경에서 중복 구현해야 한다.

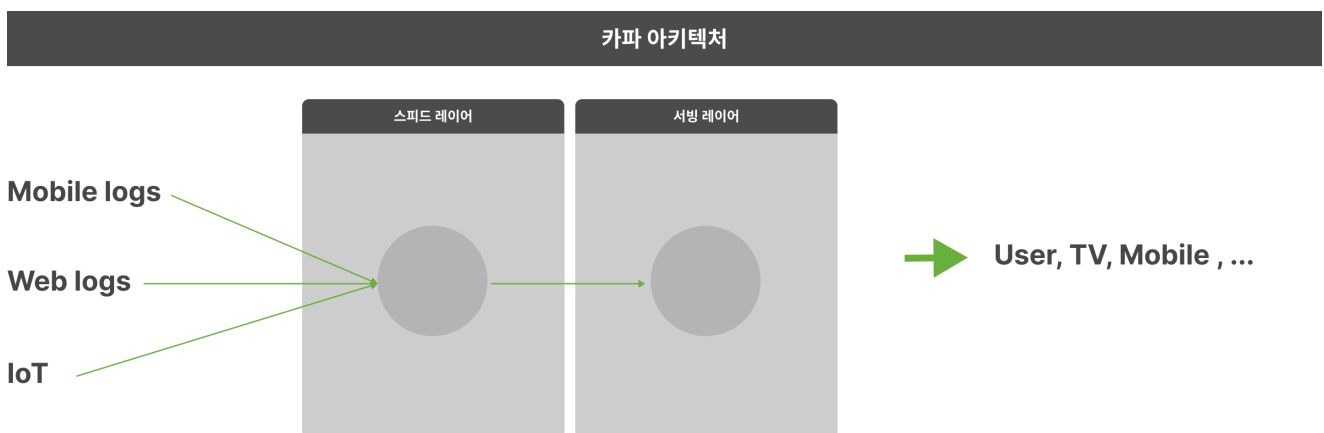
중복 구현의 문제점

1. 동일 로직이 2개라서 수정할 때 양쪽 코드를 다 고쳐야 한다
2. 미세한 차이가 생길 가능성이 있어 결과 불일치로 이어질 수 있다
3. 동일한 비즈니스 규칙을 두 환경에서 검증해야 하는데, 테스트가 복잡해진다
4. 장애 발생 시 어느 레이어에서 에러가 났는지 추적이 어려워 운영의 복잡성이 존재한다

역할은 분리되었지만, 결국 "비즈니스 로직" 수준에서 중복이 발생한다.

2.3.3 람다 아키텍처를 보완한 카파 아키텍처 (Kappa Architecture)

카파 아키텍처는 배치 레이어를 제거하고 모든 데이터를 스피드 레이어에 넣는다고 하였다.



스피드 레이어에서 데이터를 모두 처리한다. 즉, 서비스에서 생성되는 모든 종류의 데이터를 스트림 처리한다는 것이다.

배치 데이터 vs 스트림 데이터

배치 데이터

초, 분, 시간, 일 등으로 한정된 기간 단위 데이터이며 일괄 처리한다는 특징이 있다.

예시: 지난 1분간 주문한 품목, 2021년 신입생 목록

스트림 데이터

한정되지 않는 데이터로, 시작 데이터와 끝 데이터가 명확히 정해지지 않는다. 각 지점의 데이터는 보통 작은 단위로 쪼개진다.

예시: 웹 사용자 클릭 로그, 주식 정보, 사물 인터넷(IoT) 센서 데이터

2.3.3.1 배치 데이터를 어떻게 스트림 프로세스로 처리 가능한가?

모든 데이터를 로그로 바라보면 된다.

로그: 여기서는 데이터 집합을 의미한다.

이 데이터는 **지속적으로** 추가가 가능하며 데이터에 **일정한 번호**가 붙게 된다.



로그는 배치 데이터를 스트림으로 표현하기 적합하다.

로그로 표현 시 **각 시점의 배치 데이터의 변환 기록**을 시간 순으로 기록한다.

로그로 배치/스트림 데이터를 저장하고 사용하기 위해서 **변환 기록이 일정 시간 동안 삭제되면 안 되며, 지속적으로 추가되어야 한다.**

2.3.4 스트리밍 데이터 레이크 (Streaming Data Lake)

Producer → Kafka → Stream Processor(Flinks/Spark ...) → Serving Layer (DB)

: 데이터 로그 및 스트림 저장소 : 실시간 처리 : 최종 처리용 DB

데이터는 카프카, 결과는 DB — 데이터 저장과 서비스 제공을 분리한 것이다.

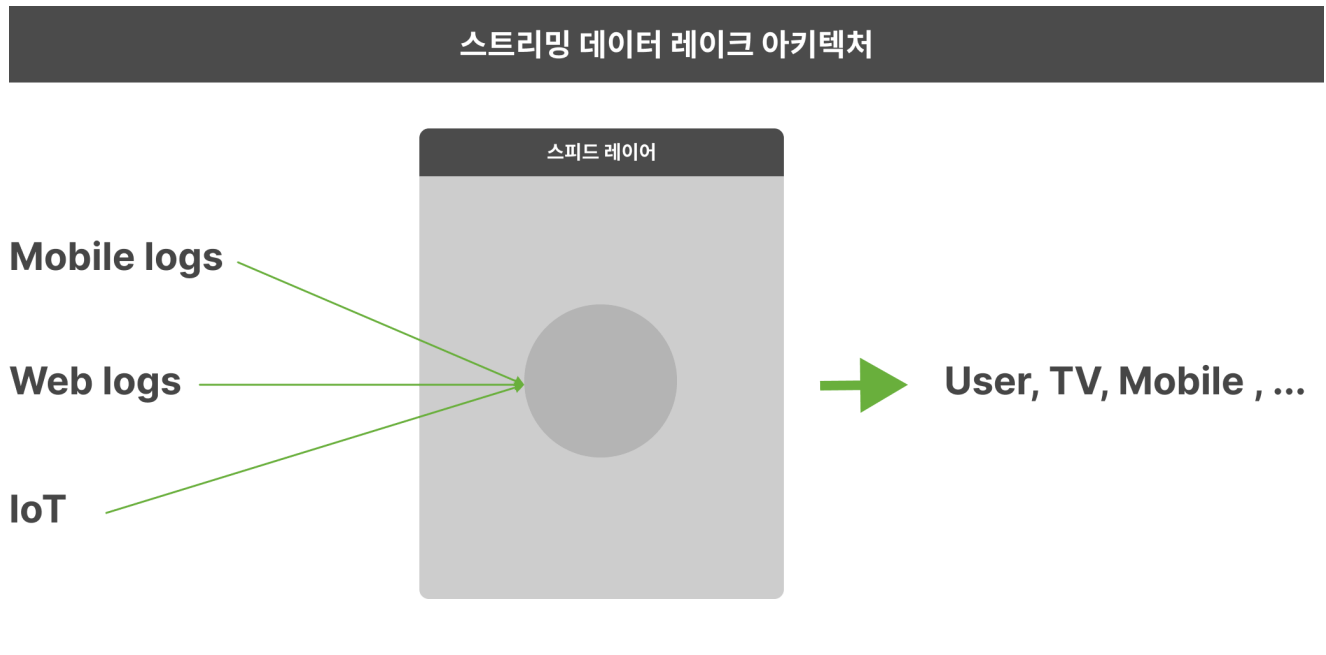
데이터를 사용하는 고객을 위해 스트림 데이터를 **서빙 레이어**에 저장한다.

서빙 레이어는 HDFS, 오브젝트 스토어와 같이 **데이터 플랫폼에서 흔히 사용되는 저장소**이다.

의문점

카프카를 통해 분석하고 프로세싱한 데이터를 다시 서빙 레이어의 저장소에 저장할 필요가 있는가?

이러한 의문에서 시작하여, 카프카로 들어온 데이터를 바로 Data Lake에 저장하고 실시간 쿼리까지 제공 하자는 뜻으로 스트리밍 데이터 레이크 아키텍처가 나오게 되었다.



2.3.5 카프카를 스트리밍 데이터 레이크로 사용할 때 유의사항

1) 자원 효율성

자주 접근하지 않는 데이터를 굳이 비싼 자원에 유지할 필요가 없다.

2) 데이터 구분

자주 사용하는 데이터만 브로커에서 사용하는 구분 작업이 필요하다.

3) 주변 플랫폼

데이터를 사용하는 곳에서 카프카의 데이터를 쿼리할 수 있는 주변 데이터 플랫폼이 필요하다.